ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 09

Bucles: repetir sin cansarse

- Qué es un bucle
- while: repetir mientras se cumpla algo
- El bucle infinito
- do…while: al menos una vez
- for: repetir un número conocido de veces
- Una primera visita a los arrays
- foreach: recorrer listas
- ¿Qué bucle uso?
- Patrones que vais a usar siempre
- break y continue
- Bucles dentro de bucles
- Bucles dentro del HTML
- Los errores típicos



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Los ordenadores son pésimos teniendo ideas originales, pero son insuperables haciendo **la misma tarea millones de veces** sin cansarse ni quejarse. Mostrar los 500 productos de una tienda, enviar un correo a cada alumno, sumar todas las líneas de una factura... Nadie escribe 500 `echo`: se escribe **uno** dentro de un **bucle**.

Hoy dedicamos la clase entera a los bucles: `while`, `do…while`, `for` y `foreach`, los patrones que vais a usar siempre y cómo meterlos dentro del HTML. Y como los bucles y las **listas** van de la mano, os haré una primera presentación de los **arrays**. En la clase 11 los veremos a fondo.

•
👌 Cómo vamos a trabajar



1
📘 Qué es un bucle



2
👍 while: repetir mientras se cumpla algo



3
⚠️ El bucle infinito



4
🔍 do...while: al menos una vez



5
🚀 for: repetir un número conocido de veces



6
😮 Una primera visita a los arrays



7
👍 foreach: recorrer listas



8
🤔 ¿Qué bucle uso?



9
🚀 Patrones que vais a usar siempre



10
⚠️ break y continue



11
😮 Bucles dentro de bucles



12
📘 Bucles dentro del HTML



13
🐞 Los errores típicos



14
🚀 Ejercicios



15

👋 Lo que tenéis que llevaros de hoy





CLASE 09

👌 Cómo vamos a trabajar

Cread la carpeta `clase09` en `php-paso-a-paso` y abrid cada archivo en `http://php-paso-a-paso.test/clase09/nombre-del-archivo.php`. Tenéis **19 ejemplos** en `ejemplos/` y las soluciones de los ejercicios en `soluciones/`.



💡 Con los bucles, un truco que funciona muy bien: antes de ejecutar, **haced vosotros de ordenador**. Coged papel y apuntad, vuelta a vuelta, cuánto vale cada variable. Después comprobad si PHP opina lo mismo. 😉



CLASE 09 · PUNTO 1

📘 Qué es un bucle

Un **bucle** es un trozo de código que se **repite**. Todos tienen las mismas piezas:

Pieza	Pregunta	Ejemplo
Inicio	¿Desde dónde empiezo?	Empiezo por el plato número 1
Condición	¿Sigo o paro?	¿Quedan platos en la pila?
Cuerpo	¿Qué hago en cada vuelta?	Fregar un plato
Avance	¿Qué cambia para acercarme al final?	Hay un plato menos

Cada repetición se llama **vuelta** o **iteración**. Si os falla alguna de las cuatro piezas, el bucle o no se ejecuta nunca o no termina nunca.



CLASE 09 · PUNTO 2

👍 while: repetir mientras se cumpla algo

Pensad en fregar platos en la cocina del restaurante de la clase 1: **mientras** quede alguno en la pila, cogéis uno, lo laváis y lo dejáis en el escurridor. No sabéis de antemano cuántos hay; simplemente paráis cuando la pila está vacía [1].

01-while-platos.php — php-paso-a-paso/clase09 — Visual Studio Code

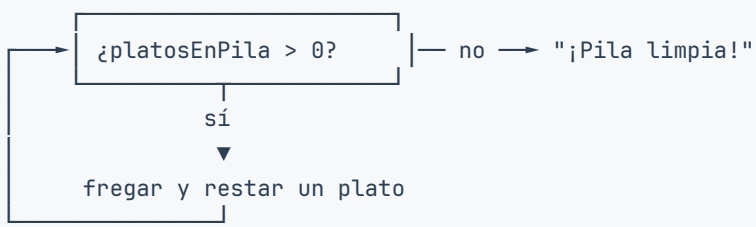
01-while-platos.php

```php
<?php
$platosEnPila = 3;

while ($platosEnPila > 0) {
    echo "Fregando un plato. Quedan {$platosEnPila}<br>";
    $platosEnPila--;   // ¡sin esta línea, no acabaríais nunca!
}
echo "¡Pila limpia!";
```

📄 Ejemplo: `01-while-platos.php`

Esquema



`while` **primero pregunta** y después hace. Si la condición es falsa desde el principio (ningún plato), el cuerpo no se ejecuta ni una vez.

También sirve para contar (`02-while-contar.php`):

ejemplo.php — php-paso-a-paso/clase09 — Visual Studio Code

ejemplo.php

```php
<?php
$numero = 1;

while ($numero ⩽ 5) {
    echo "Vuelta número $numero<br>";
    $numero++;   // $numero = $numero + 1
}

echo "Al salir, \$numero vale $numero";   // 6
```

¿Por qué vale 6 al salir? Porque el bucle para justo cuando la condición deja de cumplirse: cuando `$numero` llega a 6, `6 ⩽ 5` es falso. Haced la prueba en papel.



CLASE 09 · PUNTO 3

⚠️ El bucle infinito

Si la condición **nunca deja de cumplirse**, el bucle no termina. Es el error más famoso de la programación:

ejemplo.php — php-paso-a-paso/clase09 — Visual Studio Code

ejemplo.php

```php
<?php
$platosEnPila = 3;

while ($platosEnPila > 0) {
    echo "Fregando…";
    // ❌ nos hemos olvidado de $platosEnPila--: siempre vale 3
}
```

El servidor se queda atrapado, el navegador da vueltas y vueltas… y a los 30 segundos PHP lo corta con un error `Maximum execution time exceeded`. Si os pasa, no os asustéis: es el **mejor** momento para aprender a mirar las cuatro piezas del bucle.



🎯 Antes de ejecutar un `while`, preguntaos siempre: **¿qué línea de dentro del bucle acerca la condición a ser falsa?** Si no la encontráis, tenéis un bucle infinito.



CLASE 09 · PUNTO 4

🔍 do…while: al menos una vez

`do…while` es un `while` al revés: **primero hace y después pregunta**. Por eso el cuerpo se ejecuta **al menos una vez** [2]. Es perfecto para «tirar un dado hasta que salga un 6»: hay que tirar al menos una vez para saber qué sale.

03-do-while.php — php-paso-a-paso/clase09 — Visual Studio Code

03-do-while.php

```php
<?php
$tiradas = 0;

do {
    $dado = random_int(1, 6);
    $tiradas++;
    echo "Tirada $tiradas: $dado<br>";
} while ($dado !== 6);

echo "¡Un 6! Han hecho falta $tiradas tiradas.";
```

📄 Ejemplo: `03-do-while.php`

`random_int(min, max)` devuelve un entero al azar entre los dos valores, ambos incluidos. Recargad la página varias veces: cada vez sale distinto.



⚠️ Fijaos en el **punto y coma** después del `while (...)` final. En el `do…while` es obligatorio; en el `while` normal, no se pone.



CLASE 09 · PUNTO 5

🚀 for: repetir un número conocido de veces

Cuando **sabéis cuántas vueltas** hay que dar (del 1 al 10, las 12 cuotas de un préstamo…), el `for` reúne las piezas del bucle en una sola línea [3]:

Esquema

```
for (inicio; condición; avance) {
    cuerpo
}
```

04-for-basico.php — php-paso-a-paso/clase09 — Visual Studio Code

04-for-basico.php

```php
<?php
for ($i = 1; $i ⩽ 10; $i++) {
    echo "$i ";                          // 1 2 3 … 10
}

// De 2 en 2
for ($i = 0; $i ⩽ 20; $i += 2) {
    echo "$i ";                          // 0 2 4 … 20
}

// Cuenta atrás
for ($i = 10; $i ⩾ 0; $i--) {
    echo $i ⩵ 0 ? "¡Despegue! 🚀" : "$i... ";
}
```

📄 Ejemplo: `04-for-basico.php`

El orden en que PHP ejecuta las piezas es siempre el mismo:

Esquema

```
1. inicio ($i = 1)                      ← solo una vez
2. ¿condición? ($i ⩽ 10) → si es falsa, se acabó
3. cuerpo (echo)
4. avance ($i++)
5. vuelta al paso 2
```

La variable del contador se suele llamar `$i` (de «índice»). Es de las pocas veces en que un nombre de una letra está bien visto. 😄

El clásico de los clásicos, la tabla de multiplicar (`05-for-tabla.php`):

ejemplo.php — php-paso-a-paso/clase09 — Visual Studio Code

ejemplo.php

```php
<?php
$tabla = 7;

echo "<h2>Tabla del $tabla</h2>";
for ($i = 1; $i ⩽ 10; $i++) {
    $resultado = $tabla * $i;
    echo "$tabla × $i = $resultado<br>";
}
```

Comparad: con `while` necesitaríamos tres líneas en tres sitios distintos (inicio antes, condición arriba, avance dentro). El `for` lo deja todo a la vista en la primera línea.



CLASE 09 · PUNTO 6

😮 Una primera visita a los arrays

Los bucles brillan de verdad cuando hay que tratar **listas**: de alumnos, de productos, de notas… En PHP, una lista se guarda en un **array**: una sola variable que contiene **muchos valores** [4].

06-arrays-intro.php — php-paso-a-paso/clase09 — Visual Studio Code

06-arrays-intro.php

```php
<?php
$frutas = ["manzana", "pera", "plátano"];

// Cada elemento tiene una posición (índice) que EMPIEZA EN 0
echo "La primera: {$frutas[0]}<br>";     // manzana
echo "La tercera: {$frutas[2]}<br>";     // plátano

// Añadir al final
$frutas[] = "kiwi";

// count() dice cuántos elementos hay
echo "Hay " . count($frutas) . " frutas<br>";   // 4

// print_r() enseña el array entero (muy útil para depurar)
echo "<pre>";
print_r($frutas);
echo "</pre>";
```

📄 Ejemplo: `06-arrays-intro.php`

Imaginad una cajonera: la variable `$frutas` es el mueble, y cada cajón tiene un número pegado, **empezando por el 0**:

Esquema

```
$frutas

┌──────────┬──────────┬──────────┬──────────┐
│ manzana  │   pera   │ plátano  │   kiwi   │
└──────────┴──────────┴──────────┴──────────┘
     [0]        [1]        [2]        [3]
```



⚠️ **El primero es el 0, no el 1.** Si el array tiene 4 elementos, el último está en la posición 3 (`count() - 1`). Este despiste da muchos errores al principio.

Recorrer un array con for

Como las posiciones van de 0 a `count() - 1`, un `for` puede recorrerlas todas (`07-for-con-array.php`):

ejemplo.php — php-paso-a-paso/clase09 — Visual Studio Code

ejemplo.php

```php
<?php
$alumnos = ["Lucía", "Pablo", "Marta", "Iván"];
$total = count($alumnos);

for ($i = 0; $i < $total; $i++) {
    $numero = $i + 1;   // para las personas, la lista empieza en 1
    echo "$numero. {$alumnos[$i]}<br>";
}
```

Fijaos en el `<` (y no `⩽`): con 4 alumnos, `$i` toma los valores 0, 1, 2 y 3. Si pusierais `⩽`, intentaríais leer `$alumnos[4]`, que no existe.



CLASE 09 · PUNTO 7

👍 foreach: recorrer listas

Recorrer arrays es tan habitual que PHP tiene un bucle solo para eso: `foreach` («para cada uno»). No hay contador, ni condición, ni riesgo de salirse: recorre **todos** los elementos, uno por uno [5].

08-foreach.php — php-paso-a-paso/clase09 — Visual Studio Code

08-foreach.php

```php
<?php
$alumnos = ["Lucía", "Pablo", "Marta", "Iván"];

foreach ($alumnos as $alumno) {
    echo "Hola, $alumno<br>";
}

// Si también queremos la posición:
foreach ($alumnos as $posicion ⇒ $alumno) {
    echo "$posicion → $alumno<br>";
}
```

📄 Ejemplo: `08-foreach.php`

Se lee así: «para cada elemento de `$alumnos`, llámalo `$alumno` y haz esto». El nombre de la variable lo elegís vosotros: el plural para el array y el singular para cada elemento (`$alumnos` → `$alumno`, `$notas` → `$nota`) es una costumbre que hace el código muy fácil de leer.

Arrays con nombre: los asociativos

En lugar de posiciones numéricas, un array puede tener **claves con nombre**. Es como una carta de restaurante: cada plato con su precio. Y `foreach` nos da las dos cosas, la clave y el valor:

09-foreach-clave-valor.php — php-paso-a-paso/clase09 — Visual Studio Code

09-foreach-clave-valor.php

```php
<?php
$carrito = [
    "Teclado mecánico" => 89.99,
    "Ratón ergonómico" => 45.50,
    "Monitor 27 pulgadas" => 210.00,
];

$total = 0;

foreach ($carrito as $producto => $precio) {
    echo "$producto: " . number_format($precio, 2, ",", ".") . " €<br>";
    $total += $precio;
}

echo "<strong>Total: " . number_format($total, 2, ",", ".") . " €</strong>";
```

📄 Ejemplo: `09-foreach-clave-valor.php`

En el desarrollo web, el **95 %** de los bucles que escribiréis serán `foreach`: los productos de la tienda, los alumnos del curso, las filas que devuelve la base de datos…



CLASE 09 · PUNTO 8

🤔 ¿Qué bucle uso?

Esta tabla resuelve casi todas las dudas:

Situación	Bucle	Ejemplo
Recorrer **todos** los elementos de un array	`foreach`	Mostrar la lista de alumnos
Repetir un **número conocido** de veces	`for`	Tabla del 7, del 1 al 10, los años de un desplegable
Repetir **mientras** se cumpla algo, sin saber cuántas veces	`while`	Ahorrar hasta llegar a 1.000 €
Lo mismo, pero hay que hacerlo **al menos una vez**	`do… while`	Tirar un dado hasta que salga un 6



CLASE 09 · PUNTO 9

🚀 Patrones que vais a usar siempre

Casi todos los problemas con bucles son una combinación de **cuatro patrones**. Aprendéoslos como os aprendisteis las tablas de multiplicar.

🔢 Contador y ➕ acumulador

- Un **contador** empieza en 0 y suma **1** cada vez que pasa algo.
- Un **acumulador** empieza en 0 y suma **el valor** de cada elemento.

10-contador-acumulador.php — php-paso-a-paso/clase09 — Visual Studio Code

🐘 10-contador-acumulador.php

```php
<?php
$notas = [7, 4, 9, 5, 3, 8];
$aprobados = 0;   // contador
$suma = 0;        // acumulador

foreach ($notas as $nota) {
    $suma += $nota;
    if ($nota ⩾ 5) {
        $aprobados++;
    }
}

echo "Aprobados: $aprobados";                    // 4
echo "Media: " . round($suma / count($notas), 2); // 6
```

📄 Ejemplo: `10-contador-acumulador.php`



💡 Si lo que acumuláis es un **producto** (multiplicar), el acumulador empieza en **1**, no en 0. Si no, todo os dará 0. Lo veréis en el ejercicio del factorial.

🏆 Buscar el máximo (o el mínimo)

Suponemos que el primero es el mayor y vamos comparando con los demás. Si aparece uno mayor, ocupa su lugar:

11-buscar-maximo.php — php-paso-a-paso/clase09 — Visual Studio Code

11-buscar-maximo.php

```php
<?php
$temperaturas = [18, 23, 21, 27, 19, 25, 22];
$dias = ["lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo"];

$maxima = $temperaturas[0];
$diaMaximo = $dias[0];

for ($i = 1; $i < count($temperaturas); $i++) {
    if ($temperaturas[$i] > $maxima) {
        $maxima = $temperaturas[$i];
        $diaMaximo = $dias[$i];
    }
}

echo "La máxima fue de $maxima °C, el $diaMaximo";    // 27 °C, el jueves
```

📄 Ejemplo: `11-buscar-maximo.php`

Sí, PHP tiene la función `max()` y lo hace en una línea. Pero `max()` no os dice **qué día** fue. El patrón sí.

🚩 Buscar con una bandera

Una **bandera** es una variable booleana que empieza en `false` y se «levanta» (`true`) si encontramos lo que buscamos:

12-buscar-bandera.php — php-paso-a-paso/clase09 — Visual Studio Code

12-buscar-bandera.php

```php
<?php
$lista = ["pan", "leche", "huevos", "tomates", "aceite"];
$buscado = "huevos";
$encontrado = false;
$posicion = -1;

foreach ($lista as $indice => $producto) {
    if ($producto === $buscado) {
        $encontrado = true;
        $posicion = $indice;
        break;    // ya lo tenemos: no hace falta seguir mirando
    }
}

if ($encontrado) {
    echo "«{$buscado}» está en la posición $posicion";
} else {
    echo "«{$buscado}» no está en la lista";
}
```

📄 Ejemplo: `12-buscar-bandera.php`

🎲 Repetir hasta conseguirlo

El ordenador «piensa» un número del 1 al 20 y va probando números al azar hasta acertarlo. ¿Cuántos intentos necesitará? Ni idea. Por eso, `while`:

13-while-adivina.php — php-paso-a-paso/clase09 — Visual Studio Code

13-while-adivina.php

```php
<?php
$secreto = random_int(1, 20);
$intentos = 0;
$numero = 0;

while ($numero !== $secreto) {
    $numero = random_int(1, 20);
    $intentos++;
    echo "Intento $intentos: $numero<br>";
}
echo "¡Acertado! Ha necesitado $intentos intentos.";
```

📄 Ejemplo: `13-while-adivina.php`



CLASE 09 · PUNTO 10

⚠️ break y continue

Instrucción	Qué hace	Analogía
`continue`	Se salta **esta vuelta** y pasa a la siguiente	Saltarse una canción de la lista
`break`	**Sale** del bucle por completo [6]	Apagar la música

14-break-continue.php — php-paso-a-paso/clase09 — Visual Studio Code

14-break-continue.php

```php
<?php
// continue: del 1 al 10 sin los múltiplos de 3
for ($i = 1; $i <= 10; $i++) {
    if ($i % 3 === 0) {
        continue;
    }
    echo "$i ";         // 1 2 4 5 7 8 10
}

// break: dejamos de buscar en cuanto lo encontramos
$productos = ["pan", "leche", "huevos", "café", "fruta"];

foreach ($productos as $posicion => $producto) {
    echo "Miro la posición $posicion: $producto<br>";
    if ($producto === "huevos") {
        echo "<strong>¡Encontrado! Dejo de buscar.</strong>";
        break;
    }
}
```

📄 Ejemplo: `14-break-continue.php`

Fijaos en la salida del segundo bucle: «café» y «fruta» **no se miran**. Con 5 productos no se nota; con 5 millones de filas, `break` os ahorra mucho tiempo.



Un bucle puede ir dentro de otro. Pensad en un reloj: por cada vuelta de la aguja de las horas, la de los minutos da una vuelta **completa**.

Un ejemplo para verlo: dibujar un triángulo. El bucle de fuera recorre las **filas**; el de dentro dibuja **tantos asteriscos como el número de la fila** (`16-asteriscos.php`):

15-bucles-anidados.php — php-paso-a-paso/clase09 — Visual Studio Code

15-bucles-anidados.php

```php
<?php
$filas = 5;

echo "<pre>";
for ($fila = 1; $fila ⩽ $filas; $fila++) {
    for ($estrella = 1; $estrella ⩽ $fila; $estrella++) {
        echo "*";
    }
    echo "\n";
}
echo "</pre>";
```

Esquema

```
*
**
***
****
*****
```

El ejemplo clásico es una tabla: el de fuera recorre las **filas** y el de dentro, las **columnas** de cada fila. Aquí uso la **sintaxis alternativa** y el **echo corto**, porque estamos mezclando PHP con mucho HTML:

15-bucles-anidados.php — php-paso-a-paso/clase09 — Visual Studio Code

15-bucles-anidados.php

```php
<table>
    <?php for ($fila = 1; $fila ⩽ 10; $fila++): ?>
        <tr>
            <?php for ($col = 1; $col ⩽ 10; $col++): ?>
                <td><?= $fila * $col ?></td>
            <?php endfor; ?>
        </tr>
    <?php endfor; ?>
</table>
```

📄 Ejemplo: `15-bucles-anidados.php` (con cabeceras y algo de estilo)

¿Cuántas veces se ejecuta la línea del `<td>`? 10 filas × 10 columnas = **100 veces**. Con tres bucles anidados de 100 vueltas serían un millón: los bucles anidados son potentes, pero hay que usarlos con cabeza.



💡 Usad nombres que se entiendan (`$fila` , `$col`) en lugar de `$i` y `$j` . Con dos bucles, es muy fácil confundirse de variable.



CLASE 09 · PUNTO 12

📘 Bucles dentro del HTML

Igual que con el `if`, en las páginas usamos la sintaxis alternativa: `foreach:` … `endforeach;` y `for:` … `endfor;`. Es así como se pinta **cualquier** listado de una web (`17-lista-html.php`):

ejemplo.php — php-paso-a-paso/clase09 — Visual Studio Code

ejemplo.php

```php
<?php
$cursos = [
    ["titulo" => "PHP desde cero", "horas" => 60, "plazas" => 3],
    ["titulo" => "JavaScript moderno", "horas" => 50, "plazas" => 0],
    ["titulo" => "Diseño web", "horas" => 30, "plazas" => 12],
];
?>
<ul>
    <?php foreach ($cursos as $curso): ?>
        <li class="<?= $curso["plazas"] === 0 ? "completo" : "" ?>">
            <?= $curso["titulo"] ?> (<?= $curso["horas"] ?> h)
            <?= $curso["plazas"] === 0 ? "· completo" : "· quedan {$curso["plazas"]} plazas"
?>
        </li>
    <?php endforeach; ?>
</ul>
```

Aquí `$cursos` es un array **de arrays**: cada curso es un array asociativo con su título, sus horas y sus plazas. Os suena, ¿verdad? Es exactamente la forma que tendrán los datos cuando los saquemos de la base de datos en la clase 15. Y fijaos en cómo se juntan las dos clases: un **bucle** que recorre y un **ternario** que decide.

Y un `for` que genera las opciones de un desplegable (`18-select-anios.php`):

plantilla.php — php-paso-a-paso/clase09 — Visual Studio Code

plantilla.php

```php
<select id="anio" name="anio">
    <?php for ($anio = $anioActual; $anio >= $anioActual - 100; $anio--): ?>
        <option value="<?= $anio ?>"><?= $anio ?></option>
    <?php endfor; ?>
</select>
```

¡101 opciones con cuatro líneas! 🎉



CLASE 09 · PUNTO 13

🐞 Los errores típicos

19-errores-tipicos.php — php-paso-a-paso/clase09 — Visual Studio Code

🐘 19-errores-tipicos.php

```php
<?php
$notas = [7, 4, 9];

// 1. Salirse del array: el último índice es count() - 1
for ($i = 0; $i < count($notas); $i++) {    // ✅ con <, no con ⩽
    echo "{$notas[$i]} ";
}

// 2. Poner el acumulador DENTRO del bucle: vuelve a 0 en cada vuelta
$suma = 0;                                  // ✅ antes del bucle
foreach ($notas as $nota) {
    $suma += $nota;
}

// 3. Olvidar cambiar la condición del while: bucle infinito
$quedan = 3;
while ($quedan > 0) {
    echo "Quedan $quedan<br>";
    $quedan--;                              // ✅ sin esta línea, no acabaría nunca
}
```

📄 Ejemplo: `19-errores-tipicos.php`

Síntoma	Causa probable
`Warning: Undefined array key 3`	Os salís del array: `⩽` en lugar de `<`
El total siempre vale 0 o el último valor	El acumulador se inicializa **dentro** del bucle
La página no termina de cargar y da `Maximum execution time exceeded`	Bucle infinito: nada cambia la condición
El bucle no se ejecuta ni una vez	La condición es falsa desde el principio (¿el `>` al revés?)
Sale una vuelta de más o de menos	El famoso «error por uno»: revisad el inicio y el `<` / `⩽`



CLASE 09 · PUNTO 14

🚀 Ejercicios

Antes de cada ejercicio: **pseudocódigo** y, si el bucle tiene truco, **haced de ordenador** en papel las tres primeras vueltas.

- ☐ **01 · Pares.** Cuántos números pares hay entre 1 y 100 y cuánto suman. Debe dar 50 y 2550. → `ej01-suma-pares.php`
- ☐ **02 · FizzBuzz.** Del 1 al 30: si es múltiplo de 3, «Fizz»; de 5, «Buzz»; de los dos, «FizzBuzz»; si no, el número. → `ej02-fizzbuzz.php`
- ☐ **03 · Notas de la clase.** Con un array de alumnos (nombre y nota), calculad la media, cuántos aprueban y quién tiene la mejor nota. → `ej03-notas-clase.php`
- ☐ **04 · Triángulo.** Dibujad un triángulo de asteriscos de 5 filas con dos bucles. → `ej04-triangulo.php`
- ☐ **05 · La tabla que queráis.** Leed el número de la URL (`?tabla=9`) y mostrad su tabla de multiplicar en una tabla HTML. Si no está entre 1 y 10, un mensaje de error. → `ej05-tabla-elegida.php`
- ☐ **06 · Factorial.** Calculad 5! = 5 × 4 × 3 × 2 × 1 = 120 y mostrad la operación completa. Cuidado con el valor inicial del acumulador. → `ej06-factorial.php`
- ☐ **07 · Las cifras de un número.** Contad cuántas cifras tiene un número y escribidlo al revés (90817 → 71809). Pistas: `% 10` da la última cifra e `intdiv($n, 10)` la quita. → `ej07-digitos.php`
- ☐ **08 · Números primos.** Mostrad los primos hasta el 50 con dos bucles anidados y `break`. → `ej08-primos.php`
- ☐ **09 · Tres intentos.** Simulad lo que va escribiendo un usuario con un array de contraseñas: con un `while`, permitid como mucho 3 intentos y avisad de si entra o si se bloquea la cuenta. → `ej09-intentos-password.php`
- ☐ **10 · Lista de la compra.** Con un array de precios y otro de cantidades (con las mismas claves), mostrad cada línea y el total, saltando con `continue` los productos con cantidad 0. → `ej10-lista-compra.php`
- ☐ **11 · Calendario.** Pintad en una tabla HTML un mes de 30 días que empieza en miércoles, en filas de lunes a domingo y con el fin de semana de otro color. → `ej11-calendario.php`
- ☐ **12 · Fibonacci.** Mostrad los 15 primeros números de la sucesión de Fibonacci: cada uno es la suma de los dos anteriores (0, 1, 1, 2, 3, 5, 8…). → `ej12-fibonacci.php`
- ☐ **13 · Pirámide.** Dibujad una pirámide centrada de asteriscos de 5 filas: en cada fila, primero espacios y después asteriscos. → `ej13-piramide.php`
- ☐ **14 · La hucha.** Ahorrando 50 € al mes y con un 1 % de interés mensual sobre lo ahorrado, ¿cuántos meses hacen falta para reunir 1.000 €? → `ej14-ahorro.php`



🏆 **Reto:** en el FizzBuzz, poned la comprobación de «FizzBuzz» al final en lugar de al principio. ¿Qué pasa con el 15? Explicad por qué (pista: la clase 8 y el orden de los `elseif`).



CLASE 09 · PUNTO 15

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Las cuatro piezas	Inicio, condición, cuerpo y avance
`while`	Repite mientras se cumpla; primero pregunta
`do…while`	Primero hace y después pregunta: al menos una vez
`for`	Cuando sabéis cuántas vueltas: todo en una línea
Arrays	Listas de valores; la primera posición es la **0**
`foreach`	Recorre todos los elementos; el bucle de la web
Asociativos	Claves con nombre: `foreach ($carrito as $producto => $precio)`
Patrones	Contador, acumulador, máximo, bandera y «repetir hasta»
`break` / `continue`	Salir del bucle / saltar a la siguiente vuelta
Anidados	Por cada vuelta del de fuera, una vuelta completa del de dentro
Bucle infinito	Algo dentro del bucle tiene que acercar la condición a ser falsa

📚 Fuentes y documentación oficial

1. Manual de PHP: while
2. Manual de PHP: do-while
3. Manual de PHP: for
4. Manual de PHP: arrays
5. Manual de PHP: foreach
6. Manual de PHP: break
7. Manual de PHP: random_int